Highlights

Real-time PPE compliance monitoring on construction sites with a fine-tuned YOLO11 detector and temporal violation confirmation

Parth Mital

- A YOLO11m detector for ten PPE, violation and site-context classes
- Test mAP@0.5 of 0.8659 and mAP@0.5:0.95 of 0.6158 at 31.3 ms per image
- Test-time augmentation lowered mAP@0.5 and raised latency 2.7 times
- Five-frame temporal confirmation turns detections into video alerts
- Open-source FastAPI and React system for image and video compliance checks

Real-time PPE compliance monitoring on construction sites with a fine-tuned YOLO11 detector and temporal violation confirmation

Parth Mital

Vellore Institute of Technology, Vellore, Tamil Nadu, India

Abstract

Missing personal protective equipment (PPE) remains a leading contributor to injuries on construction sites, and manual inspection cannot watch every worker continuously. This paper presents an end-to-end PPE compliance monitoring system that couples a fine-tuned YOLO11m object detector with a rule-based compliance layer and a temporal confirmation filter for video. The detector recognises ten classes, including hardhats, safety vests and masks, their explicit absence (NO-Hardhat, NO-Safety Vest, NO-Mask), persons, safety cones, machinery and vehicles. It was trained on 2,801 images with 38,352 annotated boxes from a public construction-site dataset, re-split 80/10/10 with class stratification, for 75 epochs within a 0.70 h budget on two T4 GPUs. On the held-out test split it reaches an mAP@0.5 of 0.8659, an mAP@0.5:0.95 of 0.6158, a precision of 0.9135, a recall of 0.8053 and an inference latency of 31.3 ms per image. Test-time augmentation did not improve mAP@0.5 and nearly tripled latency. Per-class analysis shows that small or thin objects (safety cones, vests and hardhats) are the main sources of missed detections, and that 9% of true NO-Hardhat boxes are labelled as Hardhat. In the deployed system, a violation must persist for five consecutive frames before an incident is logged, which suppresses single-frame false alarms in video. The trained model is served by a FastAPI backend with a React dashboard for image and video analysis, incident review and compliance scoring.

Keywords: Construction safety, Personal protective equipment, Object detection, YOLO11, Computer vision, Compliance monitoring

1. Introduction

Construction remains one of the most hazardous industries, and failure to wear personal protective equipment (PPE) such as hardhats and high-visibility vests is a recurring contributor to injuries. Site safety officers check compliance by walking the site, which is labour-intensive, intermittent and difficult to scale across large or busy sites. Computer vision offers a way to monitor PPE use continuously from existing cameras, and reviews of computer vision for construction safety discuss the monitoring of unsafe conditions and behaviour, including PPE use, as a central application [1, 2].

Deep object detectors have made this practical. Single-stage detectors of the YOLO family [3] run in real time on commodity GPUs, and several studies have applied them to hardhat and vest detection on construction imagery [4, 5, 6]. The base study for this work, Ferdous and Ahsan [7], used the anchor-free YOLOX architecture on a dataset of colour-coded hardhats, vests and safety glasses and reported an average mAP of 89.84% for YOLOX-m. Most published systems, however, stop at detection accuracy. A site deployment also needs rules that turn boxes into compliance decisions, a way to avoid alerting on single-frame errors in video, and an interface through which safety staff can review incidents.

This paper addresses that gap with three contributions:

1. A YOLO11m detector fine-tuned for ten classes that cover PPE items, their explicit absence, persons and site context, trained under a fixed 0.70 h budget with distributed data-parallel training on two GPUs, and evaluated on stratified validation and test splits with and without test-time augmentation (TTA).
2. A detailed error analysis per class and per confusion pair, which identifies the object types and class swaps that limit compliance monitoring.
3. An open-source monitoring system in which detections pass through a compliance rule and a five-frame temporal confirmation filter before incidents are logged, served through a FastAPI backend and a React dashboard.

The rest of the paper is organised as follows. Section 2 reviews related work, Section 3 describes the system and detector, Section 4 gives the experimental setup, Section 5 reports and discusses the results, and Section 6 concludes with limitations and future work.

2. Related work

2.1. Object detection

Two-stage detectors such as Faster R-CNN [8] first propose candidate regions and then classify them, which gives high accuracy at a higher computational cost. Single-stage detectors predict boxes and classes in one pass. YOLO [3] framed detection as a single regression problem, SSD [9] added multi-scale default boxes, and RetinaNet introduced the focal loss to handle the imbalance between foreground and background examples [10]. Later YOLO versions combined these ideas with feature pyramids [11], path aggregation [12] and spatial pyramid pooling [13], and YOLOv7 showed that training-time refinements can raise accuracy without inference cost [14]. Transformer detectors such as DETR remove hand-designed components like non-maximum suppression [15], and RT-DETR brought them to real-time speeds [16]. Detectors are commonly pretrained on MS COCO [17] and evaluated with the average precision protocol introduced by PASCAL VOC [18].

2.2. Vision-based PPE detection on construction sites

Early work relied on classical vision pipelines. Park et al. detected hardhat wearing by combining separate detections of workers and hardhats in site video [19], and Mneymneh et al. isolated moving workers with a motion detection algorithm and a human classifier, then searched the worker region for hardhats with an object detector and a colour-based classifier [20]. Deep learning improved robustness. Fang et al. detected non-hardhat use with Faster R-CNN in far-field surveillance video [21], and Wu et al. released a hardhat benchmark with colour labels and an SSD detector with reverse progressive attention [5, 9]. Nath et al. compared three YOLO-based strategies for detecting workers, hardhats and vests together and reported real-time speeds [4]. Delhi et al. applied transfer learning to YOLOv3 to classify workers into four compliance categories and raise alarms for unsafe ones [22], and Wang et al. compared eight YOLO-based detectors on the CHV dataset of coloured helmets, persons and vests, where YOLOv5x reached the best mAP of 86.55% [6]. Ferdous and Ahsan introduced the CHVG dataset of 1,699 images with eight classes and found YOLOX-m to be the best of four YOLOX variants [7]. More recently, Barlybayev et al. compared YOLOv8 variants on public PPE benchmarks [23].

These studies focus on detector accuracy. Fewer describe how detections become alerts, how transient errors are filtered in video, or how the results

reach site staff. The present work uses a recent detector, reports a per-class and per-confusion error analysis, and describes the full path from frame to logged incident.

3. Proposed system and methodology

3.1. System overview

The system has four stages, shown in Figure 1. Images or videos are uploaded through a React dashboard to a FastAPI backend. Each frame passes through the YOLO11m detector, a compliance layer turns the detections into violations and person counts, and, for video, a temporal filter confirms violations before an incident is recorded with an annotated snapshot. Session metrics, incident records and the compliance score are returned to the dashboard.

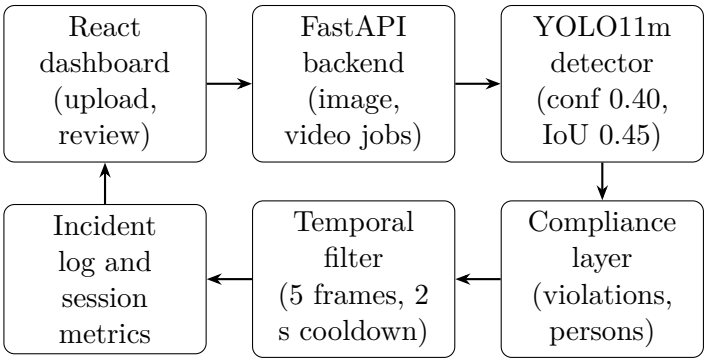


Figure 1: Processing pipeline. Photos skip the temporal filter and raise an incident directly when a violation class is detected.

3.2. Detection classes

The detector predicts the ten classes listed in Table 1. The dataset labels the absence of PPE as explicit classes (NO-Hardhat, NO-Safety Vest, NO-Mask), so violations can be detected directly instead of being inferred from a person box without a matching PPE box. In the deployed configuration, NO-Hardhat and NO-Safety Vest raise incidents, NO-Mask is reported as advisory only, and safety cones, machinery and vehicles provide site context.

Table 1: Detection classes and their role in the compliance layer.

ID	Class	Role
0	Hardhat	Required PPE
1	Mask	PPE, advisory
2	NO-Hardhat	Violation, raises an incident
3	NO-Mask	Advisory, reported only
4	NO-Safety Vest	Violation, raises an incident
5	Person	Counted for the compliance score
6	Safety Cone	Site context
7	Safety Vest	Required PPE
8	machinery	Site context
9	vehicle	Site context

3.3. Detector architecture

The detector is YOLO11m from the Ultralytics library[1], initialised from COCO-pretrained weights [17]. Its backbone stacks strided convolutions and C3k2 blocks, followed by a spatial pyramid pooling block (SPPF) in the spirit of [13] and a C2PSA attention block. The neck fuses three scales in a top-down and bottom-up path, following feature pyramid and path aggregation designs [11, 12], and an anchor-free decoupled head predicts boxes and class scores at strides 8, 16 and 32. With the ten-class head the model has 231 layers, 20,060,718 parameters and 68.3 GFLOPs at 640 px; after layer fusion for inference it has 125 layers, 20,037,742 parameters and 67.8 GFLOPs. All 649 pretrained parameter tensors were transferred; the classification head was resized from 80 to ten classes, and its Person row was initialised from the matching COCO class. Training minimises the weighted sum of a box regression loss, a classification loss and a distribution focal loss,

$$\mathcal{L} = \lambda_{\text{box}}\mathcal{L}_{\text{box}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}}\mathcal{L}_{\text{dfl}}, \tag{1}$$

with $\lambda_{\text{box}} = 7.5$, $\lambda_{\text{cls}} = 0.5$ and $\lambda_{\text{dfl}} = 1.5$.

3.4. Compliance rule and temporal confirmation

For a frame t with detected class set D_t, the set of violations is $V_t = D_t \cap \{\text{NO-Hardhat}, \text{NO-Safety Vest}\}$. The frame counts p_t persons and is

[1] `https://github.com/ultralytics/ultralytics`

marked compliant when at least one person is present and both required items are detected,

$$c_t = \begin{cases} 1 & \text{if } p_t > 0 \text{ and } \{\text{Hardhat}, \text{Safety Vest}\} \subseteq D_t, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

The session compliance score is $S = 100 \sum_t c_t / \sum_t p_t$. Because c_t is at most one per frame, S is a frame-level approximation that understates compliance in frames with several compliant workers; per-person association is left to future work (Section 6).

In video, a violation class v is confirmed at frame t only if it was detected in each of the last W frames,

$$v \in C_t \iff v \in V_{t-k} \quad \text{for all } k = 0, \ldots, W-1, \tag{3}$$

with $W = 5$. A confirmed violation creates an incident with an annotated snapshot, and a 2 s cooldown prevents repeated incidents for the same event. Single photos have no temporal context, so any detected violation raises an incident directly. The confidence and non-maximum suppression thresholds default to 0.40 and 0.45 and can be changed from the dashboard within the range 0.10 to 0.95.

4. Experimental setup

4.1. Dataset and split

The experiments use the Construction Site Safety Image Dataset distributed through Roboflow and Kaggle[2], which provides YOLO-format labels for the ten classes in Table 1. The original train, validation and test folders were pooled and re-split 80/10/10 with scikit-learn, stratified by each image's dominant class with a fixed seed of 42. Dominant-class groups with fewer than five images were merged into a background group so that every group could be split. Table 2 gives the resulting image and box counts.

Figure 2 shows the number of boxes per class in each split. Every class keeps a similar share across the three splits, which confirms that the stratification worked. The dataset is imbalanced: Person has 9,872 boxes in total, about six times as many as vehicle (1,628) or Mask (1,700).

[2] `https://www.kaggle.com/datasets/snehilsanyal/construction-site-safety-image-dataset-roboflow`

Table 2: Images and annotated boxes per split after the stratified 80/10/10 re-split.

Split	Images	Share	Boxes
Train	2,240	80.0%	30,684
Validation	280	10.0%	3,554
Test	281	10.0%	4,114
Total	2,801	100%	38,352

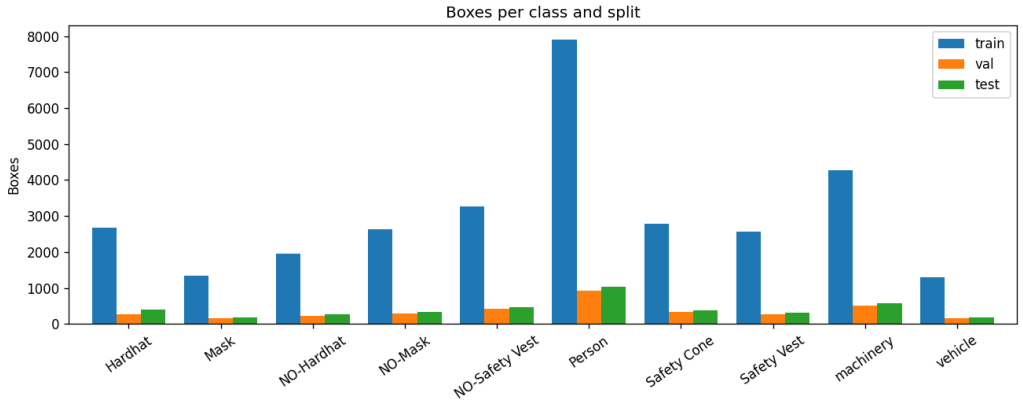


Figure 2: Annotated boxes per class in the train, validation and test splits.

4.2. Training configuration

Training ran on Kaggle with two NVIDIA Tesla T4 GPUs (15 GB each), 4 CPU cores and 33.7 GB of RAM, using Ultralytics 8.4.171, PyTorch 2.10.0 and CUDA 12.8. Both GPUs were used through PyTorch distributed data-parallel training [24] with automatic mixed precision. The run was given a wall-clock budget of 0.70 h rather than a fixed epoch count; the library fitted the epoch count and the cosine learning-rate schedule to that budget, with 200 epochs as an upper bound. Table 3 lists the main hyperparameters.

Figure 3 shows the utilisation and memory of both GPUs, sampled every 30 s during training. Utilisation of both GPUs stayed mostly between 85% and 100%, with a few short dips, and memory use held at about 8.3 GB per GPU apart from brief periodic drops, with peaks of 8,677 MB and 8,543 MB out of 15,360 MB.

Table 3: Training hyperparameters, as recorded in the training log (LR: learning rate).

Parameter	Value	Parameter	Value
Input size	640 px	Optimiser	SGD
Batch size	32 (16 per GPU)	Momentum	0.937
Time budget	0.70 h	Initial LR	0.01
Epoch limit	200 (upper bound)	Final LR factor	0.01
Warm-up	3 epochs	LR schedule	Cosine
Mosaic	1.0	Weight decay	0.0005
Mosaic off	Last 10 epochs	Patience	100 epochs
Mixup / copy-paste	0.1 / 0.1	Translation / scale	0.1 / 0.5
Rotation / shear	5.0 / 2.0 degrees	HSV (h, s, v)	0.015, 0.7, 0.4
Horizontal flip	0.5	Image cache	RAM
Mixed precision	Yes (AMP)	Seed	42

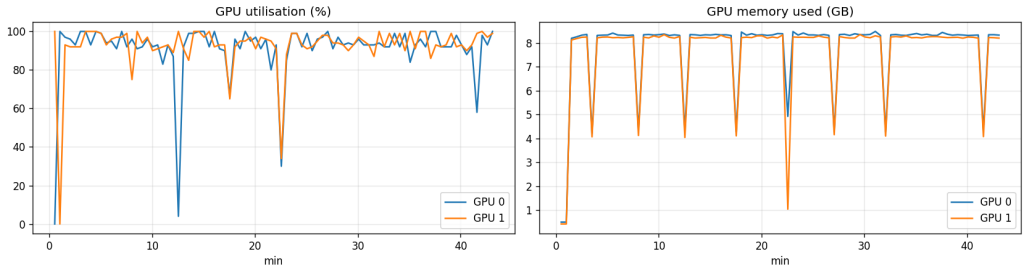


Figure 3: Utilisation and memory use of the two T4 GPUs during training.

4.3. Evaluation protocol

The checkpoint with the best validation fitness (`best.pt`) was evaluated on the validation and test splits at 640 px with half precision and a batch size of 16, and the test split was evaluated again with flip and scale TTA. For each class, average precision (AP) is the area under the precision-recall curve, where a prediction is a true positive when its intersection over union (IoU) with a ground-truth box of the same class reaches the threshold [18]. mAP@0.5 averages AP over classes at an IoU of 0.5, and mAP@0.5:0.95 also averages over IoU thresholds from 0.5 to 0.95 in steps of 0.05, following COCO [17]. Precision P and recall R are reported at the confidence that maximises the mean F1 score, and the summary F1 is

$$F_1 = \frac{2PR}{P + R}. \tag{4}$$

Latency is the model inference time per image reported by the evaluator on one T4.

5. Results and discussion

5.1. Training behaviour

The run completed 75 epochs in 0.701 h. Figure 4 plots the losses, precision, recall, mAP and learning rate per epoch. Training losses fall steadily, with a visible step down at epoch 66 when mosaic augmentation is switched off. Validation losses rise briefly in epochs 3 to 6 just after warm-up, then fall and level off after about epoch 60. The best epoch was 72, with a validation mAP@0.5 of 0.8692 and mAP@0.5:0.95 of 0.6375, and from epoch 68 onward validation mAP@0.5:0.95 stayed between 0.632 and 0.638, so training ended on a plateau rather than being cut off while still improving. The learning rate decays from its warm-up peak to 1.04×10^{-4} at epoch 75.

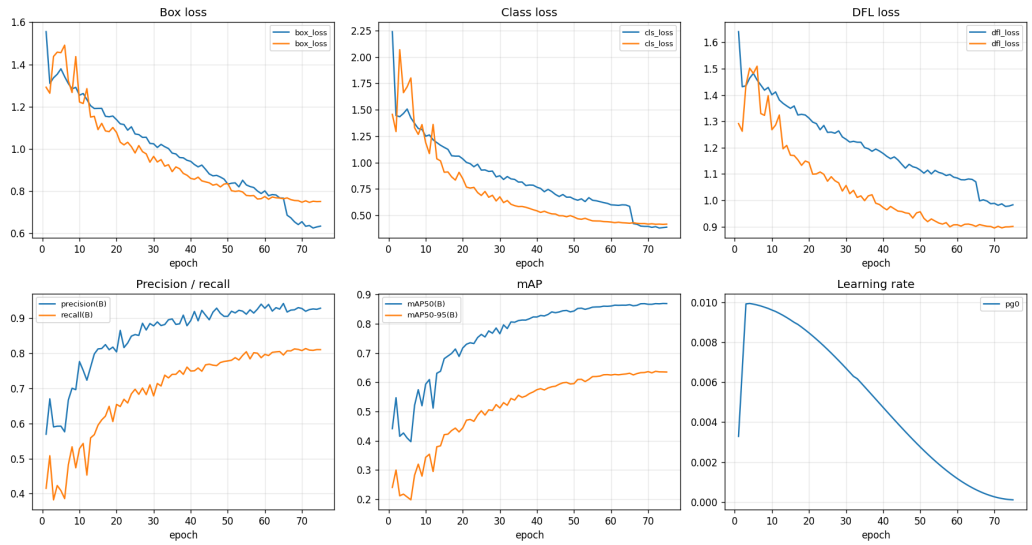


Figure 4: Per-epoch box, classification and distribution focal losses (training in blue, validation in orange), validation precision and recall, validation mAP, and learning rate.

5.2. Overall detection accuracy

Table 4 summarises accuracy and latency. On the held-out test split, the model reaches an mAP@0.5 of 0.8659 and an mAP@0.5:0.95 of 0.6158,

close to the validation values of 0.8679 and 0.6368. The small gap between validation and test suggests that the checkpoint selection did not overfit the validation split. Precision (0.9135) is clearly higher than recall (0.8053) on the test split, so the model's errors are mostly missed objects rather than false alarms at the operating point used for these figures.

Table 4: Overall accuracy and inference latency of `best.pt` (640 px, fp16, batch 16, one T4).

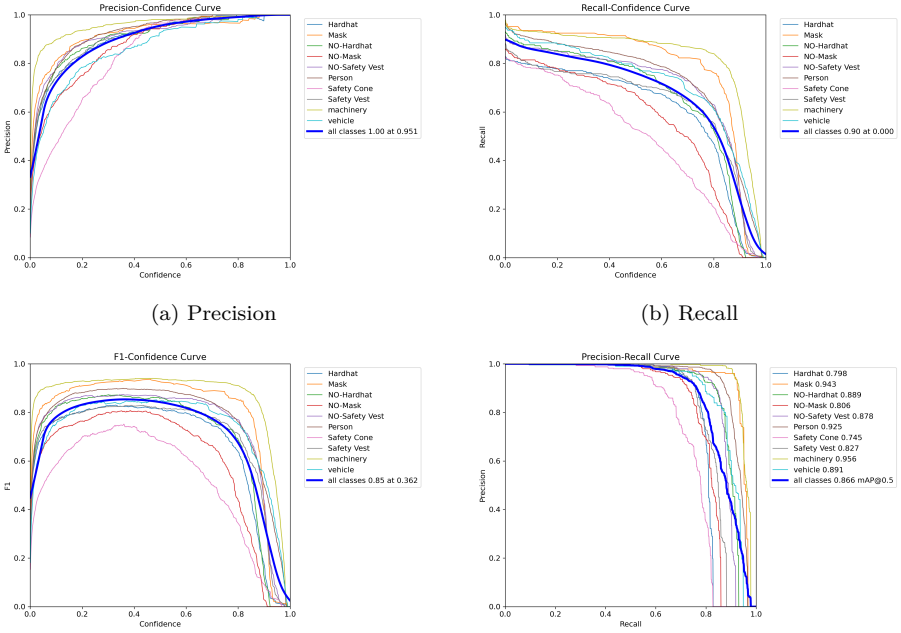
Split	mAP@0.5	mAP@0.5:0.95	Precision	Recall	F1	ms/img
Validation	0.8679	0.6368	0.9236	0.8084	0.8621	32.79
Test	0.8659	0.6158	0.9135	0.8053	0.8560	31.32
Test with TTA	0.8603	0.6206	0.9027	0.7853	0.8399	85.98

TTA lowered mAP@0.5 from 0.8659 to 0.8603, precision from 0.9135 to 0.9027 and recall from 0.8053 to 0.7853, and raised only mAP@0.5:0.95, from 0.6158 to 0.6206. It took 85.98 ms per image, 2.7 times the 31.32 ms of plain inference. For a monitoring system that must keep up with video, TTA is therefore not worthwhile, and the deployed backend does not use it. In the final validation inside the training run, preprocessing, inference and postprocessing took 1.3 ms, 15.0 ms and 6.2 ms per image. Predicting all 281 test images, with 4,002 boxes drawn, ran at 35.9 images per second end to end.

Figure 5 shows the precision, recall, F1 and precision-recall curves for the test split; the validation-split curves are in Figure A.9 in the appendix. On the test split, the all-class F1 curve peaks at 0.85 at a confidence of 0.362 and stays close to that value over a broad plateau, so the deployed default of 0.40 lies in a stable region rather than on a sharp optimum.

5.3. Per-class accuracy

Table 5 breaks accuracy down by class for the validation and test splits.



(a) Precision (b) Recall

(c) F1 (d) Precision-recall

Figure 5: Held-out test split (281 images): precision, recall and F1 against confidence, and the precision-recall curve per class.

Table 5: Per-class AP@0.5 (AP50), AP@0.5:0.95 (AP), precision (P) and recall (R) on the validation and test splits.

	Validation				Test			
Class	AP50	AP	P	R	AP50	AP	P	R
Hardhat	0.8592	0.6170	0.9339	0.8188	0.7975	0.5420	0.9183	0.7473
Mask	0.9763	0.7667	0.9609	0.9345	0.9432	0.7137	0.9411	0.9194
NO-Hardhat	0.8676	0.5992	0.9188	0.8043	0.8894	0.5813	0.9217	0.8179
NO-Mask	0.8249	0.5129	0.9209	0.7433	0.8055	0.4587	0.8878	0.7346
NO-Safety Vest	0.9034	0.6753	0.9161	0.8535	0.8782	0.6445	0.9318	0.8194
Person	0.9290	0.7275	0.9638	0.8581	0.9250	0.7124	0.9459	0.8540
Safety Cone	0.7027	0.3969	0.8468	0.6172	0.7447	0.3963	0.8583	0.6641
Safety Vest	0.8317	0.6284	0.9388	0.7820	0.8273	0.5983	0.9103	0.7572
machinery	0.9555	0.8290	0.9520	0.9109	0.9565	0.8325	0.9663	0.9065
vehicle	0.8288	0.6155	0.8839	0.7612	0.8914	0.6781	0.8538	0.8324

Machinery (test AP@0.5 0.9565), Mask (0.9432) and Person (0.9250) are

detected best. Safety Cone is the weakest class on every split, with a test AP@0.5 of 0.7447 and an AP@0.5:0.95 of only 0.3963, which reflects how small and thin cones usually are in site images. Hardhat drops from 0.8592 on validation to 0.7975 on test, the largest validation-to-test drop of any class, and NO-Mask (0.8055) and Safety Vest (0.8273) are also below the mean. For compliance monitoring the two incident classes matter most: NO-Hardhat reaches a test AP@0.5 of 0.8894 and NO-Safety Vest 0.8782, both above the overall mean.

Figure 6 compares per-class AP@0.5 across the validation split, the test split and the test split with TTA. TTA changes most classes by less than 0.01; the largest change is a drop for Safety Cone, from 0.7447 to 0.7146, so augmentation at inference does not help the weakest class either.

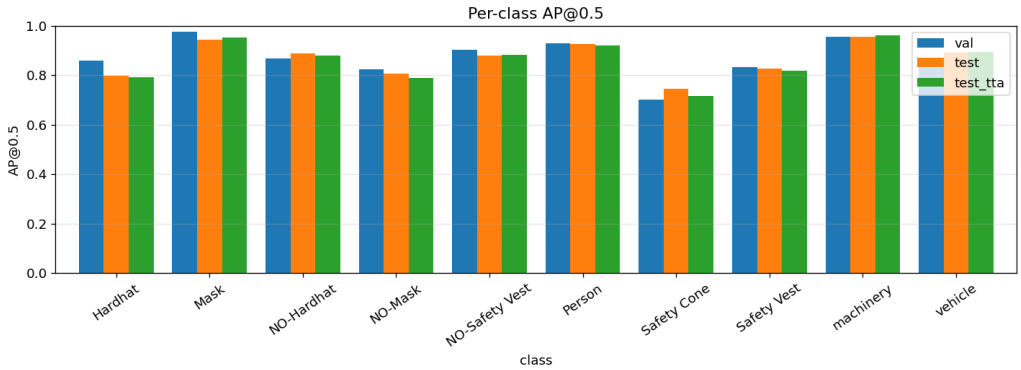
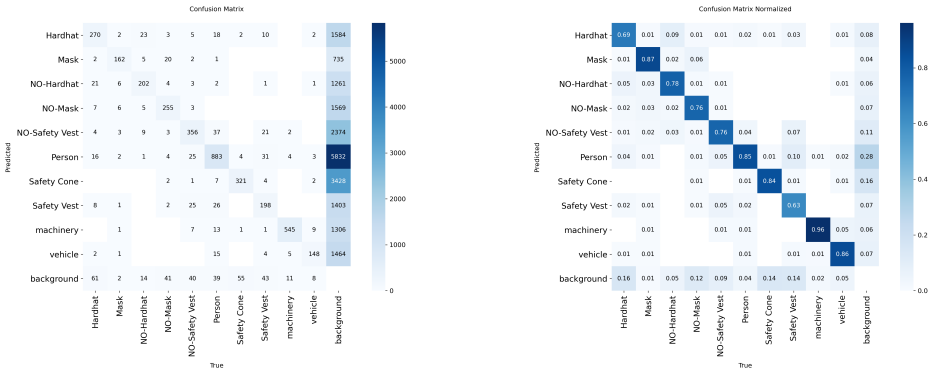


Figure 6: Per-class AP@0.5 on the validation split, the test split and the test split with TTA (`test_tta`).

5.4. Error analysis

The confusion matrices in Figure 7 show where errors come from on the test split; the validation matrices are in Figure A.10. In the normalised test matrix, the diagonal ranges from 0.63 for Safety Vest to 0.96 for machinery. The largest single error is missed detection: 16% of true Hardhat boxes and 14% each of true Safety Cone and Safety Vest boxes are not detected at all (predicted as background), and 12% of NO-Mask and 9% of NO-Safety Vest boxes are missed. Among class swaps, 10% of true Safety Vest boxes are predicted as Person, which is consistent with a vest being a region inside a person box.



(a) Counts (b) Normalised by true class

Figure 7: Confusion matrices on the held-out test split.

Two swaps matter directly for safety. First, 9% of true NO-Hardhat boxes are predicted as Hardhat, which turns a violation into an apparent compliance; this is the most consequential error for the incident pipeline. Second, 7% of true Safety Vest boxes are predicted as NO-Safety Vest, which would raise false alarms. On the background side, 28% of false positives against background are predicted as Person and 16% as Safety Cone. The temporal filter in Equation (3) is meant to suppress such errors when they are transient: a misclassification that appears in fewer than five consecutive frames raises no incident. Errors that persist across frames, such as a NO-Hardhat worker consistently labelled as Hardhat, are not corrected by the filter.

Figure 8 shows predictions on six test images chosen with a fixed seed. Workers, hardhats, vests and machinery are localised at several scales. Several dataset images are themselves composites of four photographs, as the examples show.

5.5. Comparison with prior work

Direct numerical comparison with earlier studies is not possible because they use different datasets, class sets and splits. The base study [7] reported an average mAP of 89.84% for YOLOX-m on its eight-class CHVG dataset of 1,699 images, while this work reports an mAP@0.5 of 0.8659 on a different ten-class dataset of 2,801 images that includes explicit violation classes and site-context classes. The two results are of a similar order, but the difference cannot be attributed to the detector alone. What the present work adds is

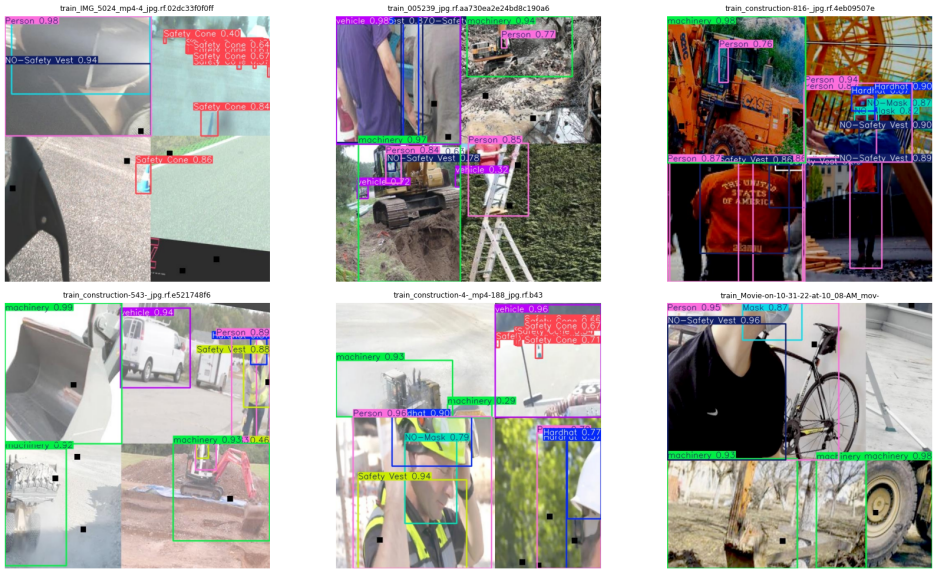


Figure 8: Predicted boxes on six test images selected with seed 42.

the path from detections to incidents: explicit violation classes, a compliance rule, temporal confirmation for video and a working interface, which detection-only studies do not evaluate.

5.6. Deployment

The trained weights are served by a FastAPI backend that exposes endpoints for image analysis, asynchronous video jobs with progress reporting, incident listing, session metrics, threshold configuration and model reload, and a React dashboard presents the results. The model was also exported to ONNX (opset 17, 80.5 MB) with an output tensor of shape $1 \times 14 \times 8400$, four box values and ten class scores for each of 8,400 candidate locations, for runtimes other than PyTorch. The complete notebook, from data loading to export, took 44.88 min, of which training took 43.49 min.

6. Conclusion and future work

This paper presented a PPE compliance monitoring system built around a YOLO11m detector fine-tuned for ten construction-site classes. Trained for 75 epochs within a 0.70 h budget on two T4 GPUs, the detector reaches an mAP@0.5 of 0.8659 and an mAP@0.5:0.95 of 0.6158 on a held-out test split

at 31.3 ms per image, with closely matching validation results. Test-time augmentation did not help and nearly tripled latency. The error analysis shows that small and thin objects (safety cones, vests and hardhats) account for most missed detections, and that confusing NO-Hardhat with Hardhat is the most safety-critical error. A five-frame temporal confirmation filter and a compliance rule turn detections into incidents, and the system is available as an open-source FastAPI and React application.

The work has several limitations. All metrics come from one training run on one dataset split, without cross-validation or an external test set, so accuracy on other sites, cameras, weather and lighting has not been measured. The compliance score is a frame-level approximation that does not associate PPE items with individual workers. The temporal filter and alert cooldown were set by design rather than tuned on annotated video, and the false-alarm reduction they provide has not been measured. Video jobs are held in memory, so a backend restart loses running jobs, and model inference is guarded by a lock, so concurrent jobs share a single inference stream.

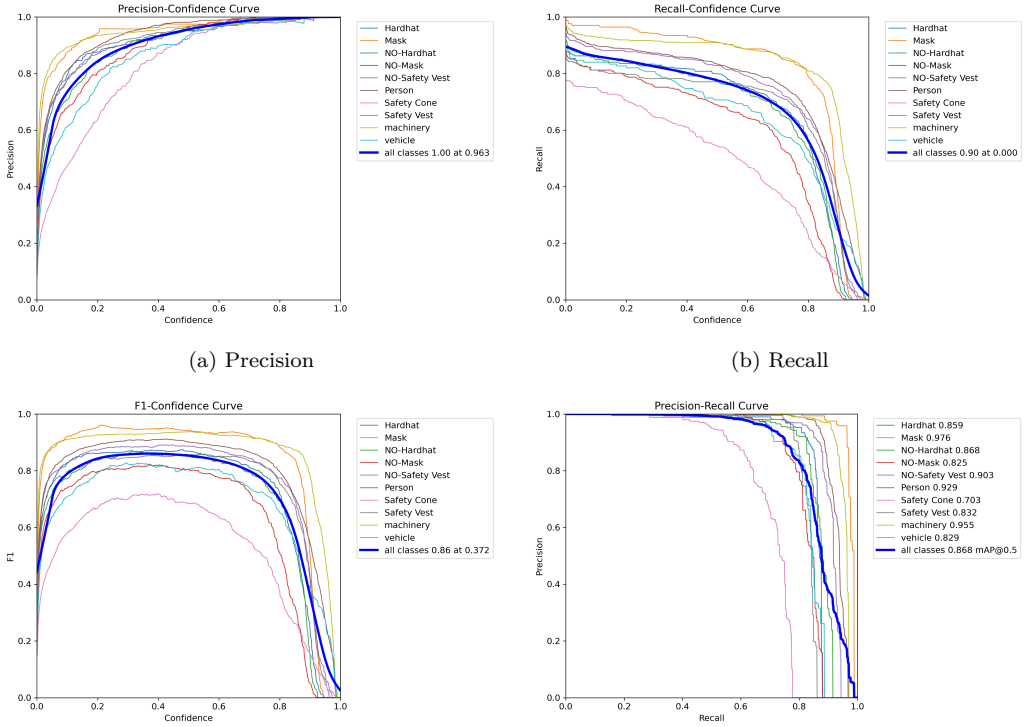
Future work will address these points: associating PPE boxes with person boxes and tracking workers across frames to compute per-worker compliance; evaluating on footage from other sites and under adverse conditions; tuning the temporal window and thresholds on annotated video to quantify false-alarm reduction; targeting the weak classes with higher input resolution or small-object augmentation; and comparing YOLO11 with transformer-based real-time detectors such as RT-DETR [16] on the same split.

Data and code availability

The source code, training notebook, trained weights and all result files are available at `https://github.com/parthmital/PPE-Compliance-Monitor`. The dataset is publicly available on Kaggle at the address given in Section 4.

Appendix A. Additional evaluation plots

This appendix collects the remaining plots produced by the training and evaluation runs. Figures A.9 and A.10 show the curves and confusion matrices of the validation split. Figure A.11 shows the per-epoch summary drawn by Ultralytics during training. Figures A.12 and A.13 show the curves and confusion matrices from the validation that ran at the end of training, on the same validation split as Figures A.9 and A.10 but from a separate call. Figures A.14 and A.15 show the test split evaluated with TTA.



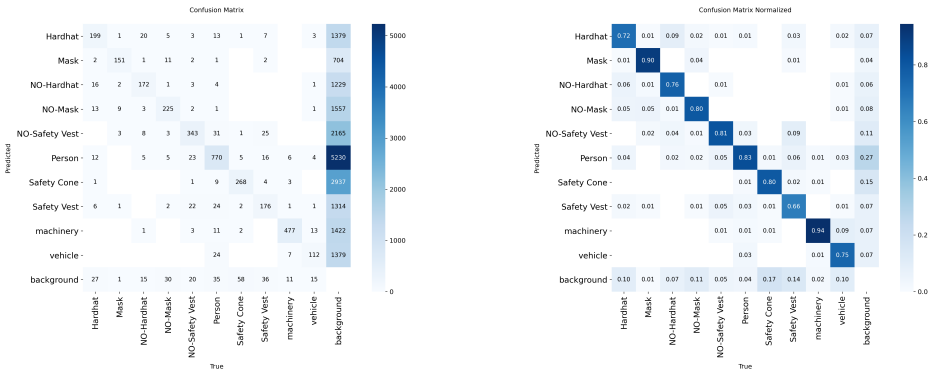
(a) Precision

(b) Recall

(c) F1

(d) Precision-recall

Figure A.9: Validation split (280 images): precision, recall and F1 against confidence, and the precision-recall curve per class.



(a) Counts

(b) Normalised by true class

Figure A.10: Confusion matrices on the validation split.

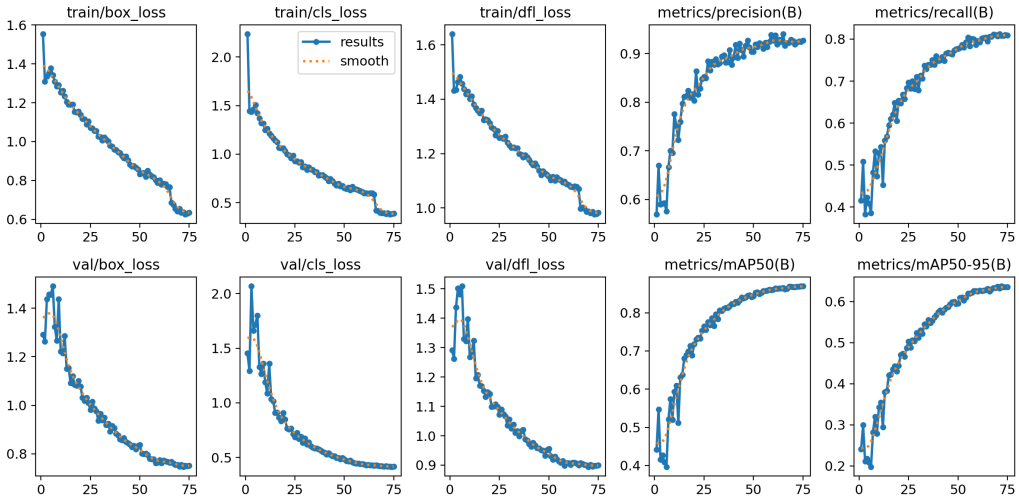
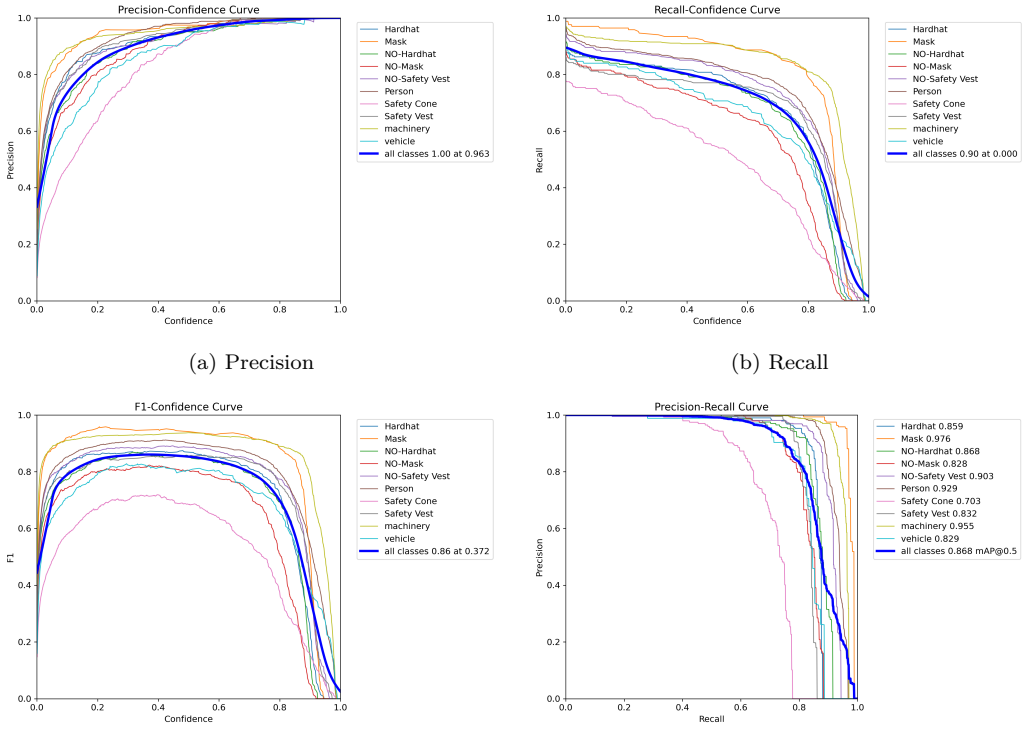


Figure A.11: Per-epoch losses and metrics as plotted by Ultralytics during training.



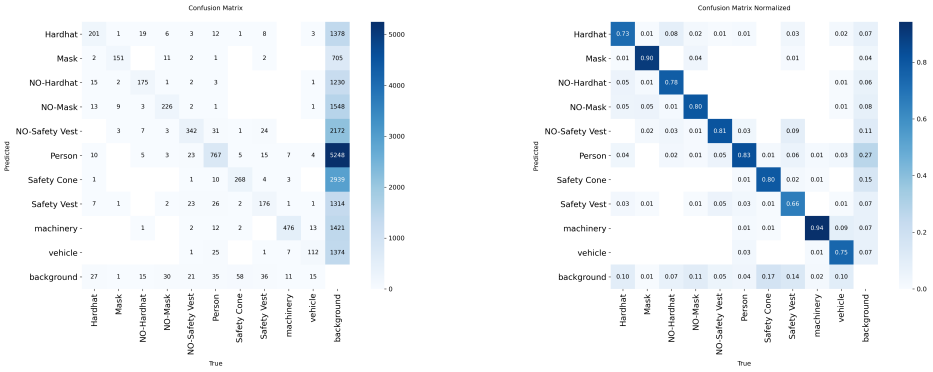
(a) Precision

(b) Recall

(c) F1

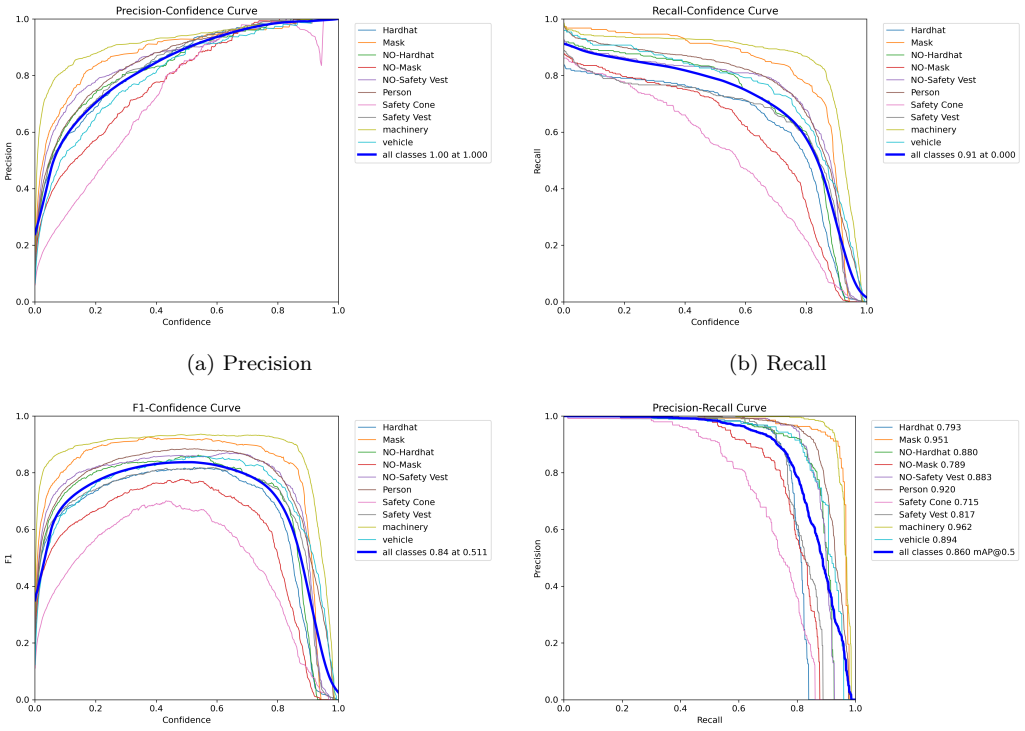
(d) Precision-recall

Figure A.12: Curves from the end-of-training validation of `best.pt`.



(a) Counts (b) Normalised by true class

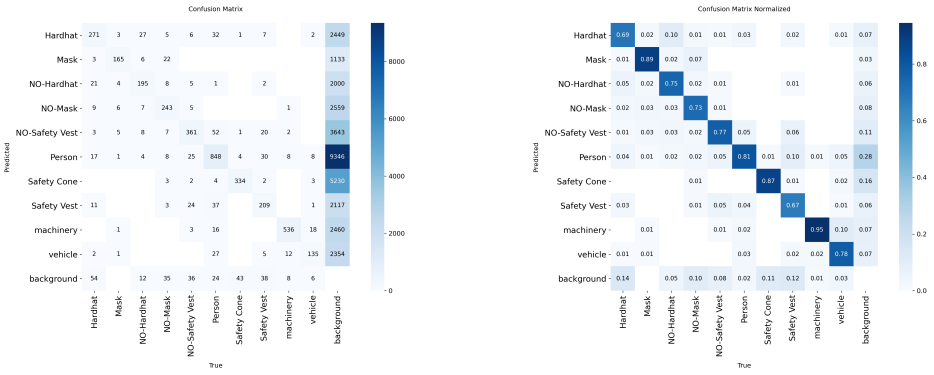
Figure A.13: Confusion matrices from the end-of-training validation of `best.pt`.



(a) Precision (b) Recall

(c) F1 (d) Precision-recall

Figure A.14: Curves on the test split with flip and scale TTA.



(a) Counts

(b) Normalised by true class

Figure A.15: Confusion matrices on the test split with flip and scale TTA.

References

[1] J. Seo, S. Han, S. Lee, H. Kim, Computer vision techniques for construction safety and health monitoring, Advanced Engineering Informatics 29 (2) (2015) 239–251. doi:10.1016/j.aei.2015.02.001.

[2] W. Fang, P. E. D. Love, H. Luo, L. Ding, Computer vision for behaviour-based safety in construction: A review and future directions, Advanced Engineering Informatics 43 (2020) 100980. doi:10.1016/j.aei.2019.100980.

[3] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788. doi:10.1109/CVPR.2016.91.

[4] N. D. Nath, A. H. Behzadan, S. G. Paal, Deep learning for site safety: Real-time detection of personal protective equipment, Automation in Construction 112 (2020) 103085. doi:10.1016/j.autcon.2020.103085.

[5] J. Wu, N. Cai, W. Chen, H. Wang, G. Wang, Automatic detection of hardhats worn by construction personnel: A deep learning approach and benchmark dataset, Automation in Construction 106 (2019) 102894. doi:10.1016/j.autcon.2019.102894.

[6] Z. Wang, Y. Wu, L. Yang, A. Thirunavukarasu, C. Evison, Y. Zhao, Fast personal protective equipment detection for real construction sites using deep learning approaches, Sensors 21 (10) (2021) 3478. doi:10.3390/s21103478.

[7] M. Ferdous, S. M. M. Ahsan, PPE detector: a YOLO-based architecture to detect personal protective equipment (PPE) for construction sites, PeerJ Computer Science 8 (2022) e999. doi:10.7717/peerj-cs.999.

[8] S. Ren, K. He, R. Girshick, J. Sun, Faster R-CNN: Towards real-time object detection with region proposal networks, IEEE Transactions on Pattern Analysis and Machine Intelligence 39 (6) (2017) 1137–1149. doi:10.1109/TPAMI.2016.2577031.

[9] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, A. C. Berg, SSD: Single shot MultiBox detector, in: Computer Vision – ECCV 2016, 2016, pp. 21–37. doi:10.1007/978-3-319-46448-0_2.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár, Focal loss for dense object detection, in: 2017 IEEE International Conference on Computer Vision (ICCV), 2017, pp. 2999–3007. doi:10.1109/ICCV.2017.324.

[11] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie, Feature pyramid networks for object detection, in: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 936–944. doi:10.1109/CVPR.2017.106.

[12] S. Liu, L. Qi, H. Qin, J. Shi, J. Jia, Path aggregation network for instance segmentation, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 8759–8768. doi:10.1109/CVPR.2018.00913.

[13] K. He, X. Zhang, S. Ren, J. Sun, Spatial pyramid pooling in deep convolutional networks for visual recognition, IEEE Transactions on Pattern Analysis and Machine Intelligence 37 (9) (2015) 1904–1916. doi:10.1109/TPAMI.2015.2389824.

[14] C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in:

2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2023, pp. 7464–7475. doi:10.1109/CVPR52729.2023.00721.

[15] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko, End-to-end object detection with transformers, in: Computer Vision – ECCV 2020, 2020, pp. 213–229. doi:10.1007/978-3-030-58452-8_13.

[16] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, J. Chen, DETRs beat YOLOs on real-time object detection, in: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024, pp. 16965–16974. doi:10.1109/CVPR52733.2024.01605.

[17] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, C. L. Zitnick, Microsoft COCO: Common objects in context, in: Computer Vision – ECCV 2014, 2014, pp. 740–755. doi:10.1007/978-3-319-10602-1_48.

[18] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, A. Zisserman, The Pascal visual object classes (VOC) challenge, International Journal of Computer Vision 88 (2) (2010) 303–338. doi:10.1007/s11263-009-0275-4.

[19] M.-W. Park, N. Elsafty, Z. Zhu, Hardhat-wearing detection for enhancing on-site safety of construction workers, Journal of Construction Engineering and Management 141 (9) (2015) 04015024. doi:10.1061/(ASCE)CO.1943-7862.0000974.

[20] B. E. Mneymneh, M. Abbas, H. Khoury, Vision-based framework for intelligent monitoring of hardhat wearing on construction sites, Journal of Computing in Civil Engineering 33 (2) (2019) 04018066. doi:10.1061/(ASCE)CP.1943-5487.0000813.

[21] Q. Fang, H. Li, X. Luo, L. Ding, H. Luo, T. M. Rose, W. An, Detecting non-hardhat-use by a deep learning method from far-field surveillance videos, Automation in Construction 85 (2018) 1–9. doi:10.1016/j.autcon.2017.09.018.

[22] V. S. K. Delhi, R. Sankarlal, A. Thomas, Detection of personal protective equipment (PPE) compliance on construction site using computer

vision based deep learning techniques, Frontiers in Built Environment 6 (2020) 136. doi:10.3389/fbuil.2020.00136.

[23] A. Barlybayev, N. Amangeldy, B. Kurmetbek, I. Krak, B. Razakhova, N. Tursynova, R. Turebayeva, Personal protective equipment detection using YOLOv8 architecture on object detection benchmark datasets: a comparative study, Cogent Engineering 11 (1) (2024) 2333209. doi:10.1080/23311916.2024.2333209.

[24] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, S. Chintala, PyTorch distributed: experiences on accelerating data parallel training, Proceedings of the VLDB Endowment 13 (12) (2020) 3005–3018. doi:10.14778/3415478.3415530.